

Module-3

TOPIC 1: Merge Sort

1. Rank Student by Marks.

```
#include <stdio.h>
void merge(int arr[], int l, int m, int r)
{
    int i, j, k;
    int n1 = m - l + 1;
    int n2 = r - m;

    int L[n1], R[n2];

    for (i = 0; i < n1; i++)
        L[i] = arr[l + i];

    for (j = 0; j < n2; j++)
        R[j] = arr[m + 1 + j];

    i = 0;
    j = 0;
    k = l;

    while (i < n1 && j < n2)
    {
        if (L[i] <= R[j])
            arr[k++] = L[i++];
    }
```


else

arr[k++] = R[j++];
}

while (j < n1)

arr[k++] = L[i++];

while (j < n2)

arr[k++] = R[j++];

}

void mergesort(int arr[], int l, int r)

{

if (l < r)

{

int m = (l + r) / 2;

mergesort(arr, l, m);

mergesort(arr, m+1, r);

mergeSort(arr, l, m, r);

}

}

int main()

{

int n, i;

~~scanf("%d", &n);~~

printf("Enter number of student:");

scanf("%d", &n);

int arr[n];


```
printf("Enter marks: \n");  
for (i = 0; i < n; i++)  
    scanf("%d", &arr[i]);
```

```
mergeSort(arr, 0, n-1);
```

```
for (i = 0; i < n; i++)  
    printf("%d", arr[i]);
```

```
return 0;
```

```
}
```


C Mergesort1.c > merge(int [], int, int, int)

```
1  #include <stdio.h>
2
3  // Function to merge two sorted subarrays
4  void merge(int arr[], int l, int m, int r)
5  {
6      int i, j, k;
7
8      // Size of left and right subarrays
9      int n1 = m - l + 1;
10     int n2 = r - m;
11
12     // Temporary arrays
13     int L[n1], R[n2];
14
15     // Copy data into temporary arrays
16     for(i = 0; i < n1; i++)
17         L[i] = arr[l + i];
18
19     for(j = 0; j < n2; j++)
20         R[j] = arr[m + 1 + j];
21
22     i = 0;
23     j = 0;
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

PS C:\Users\sanskriti shukla\Documents\C programs\Module 3> cd

gcc Mergesort1.c -o Mergesort1 } ; if (\$?) { .\Mergesort1 }

enter number of students: 6

enter marks:

78

45

92

34

67

81

Sorted Marks: 34 45 67 78 81 92

PS C:\Users\sanskriti shukla\Documents\C programs\Module 3>

2. Organizing Product Prices.

```
#include <stdio.h>
void merge (int arr[], int left, int mid, int right)
{
    int i, j, k;
    int n1 = mid - left + 1;
    int n2 = right - mid;

    int L[n1], R[n2];

    for (i = 0; i < n1; i++)
        L[i] = arr[left + i];

    for (j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];

    i = 0;
    j = 0;
    k = left;

    // merge the temporary arrays
    while (i < n1 && j < n2)
    {
        if (L[i] <= R[j])
            arr[k++] = L[i++];
        else
            arr[k++] = R[j++];
    }
}
```



```
// Copy remaining Element  
while (i < n1)  
    arr[k++] = L[i++];
```

```
while (j < n2)  
    arr[k++] = R[j++];  
}
```

```
// Recursive Merge Sort function  
void mergeSort(int arr[], int left, int right)  
{  
    if (left < right)  
    {  
        int mid = (left + right) / 2;  
  
        mergeSort(arr, left, mid);  
        mergeSort(arr, mid + 1, right);  
        mergeSort(arr, left, mid, right);  
    }  
}
```

```
int main()  
{
```

```
    int n, i;
```

```
    printf("Enter number of products:");  
    scanf("%d", &n);
```

```
    int arr[n];
```

```
    printf("Enter prices: \n");
```



```
for (i = 0; i < n; i++)  
    scanf ("%d", &arr[i]);
```

```
mergeSort (arr, 0, n-1);
```

```
printf ("Sorted prices : ");  
for (i = 0; i < n; i++)  
    printf ("%d", arr[i]);
```

```
return 0;
```

```
}
```



```
Mergesort2.c main()
1  #include <stdio.h>
2  // Function to merge two sorted subarrays
3  void merge(int arr[], int left, int mid, int right)
4  {
5      int i, j, k;
6      int n1 = mid - left + 1;
7      int n2 = right - mid;
8
9      int L[n1], R[n2];
10
11     // Copy data into temporary arrays
12     for(i = 0; i < n1; i++)
13         L[i] = arr[left + i];
14
15     for(j = 0; j < n2; j++)
16         R[j] = arr[mid + 1 + j];
17
18     i = 0;
19     j = 0;
20     k = left;
21
22     // Merge the temporary arrays
23     while(i < n1 && j < n2)
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

PS C:\Users\sanskriti shukla\Documents\C programs\Module 3> cd "

gcc Mergesort2.c -o Mergesort2 } ; if (\$?) { .\Mergesort2 }

enter number of products: 5

enter prices:

2500

1500

4000

3000

1000

Sorted Prices: 1000 1500 2500 3000 4000

3. Sort Employee Records by Salary

```
#include <stdio.h>
```

```
void merge(int arr[], int leftlength, int mid, int right)
```

```
{  
    int i, j, k;  
    int n1 = mid - left + 1;  
    int n2 = right - mid;
```

```
    int L[n1], R[n2];
```

```
    for(i = 0; i < n1; i++)  
        L[i] = arr[left + i];
```

```
    for(j = 0; j < n2; j++)  
        R[j] = arr[mid + 1 + j];
```

```
    i = 0;
```

```
    j = 0;
```

```
    k = left;
```

```
    while(i < n1 && j < n2)  
    {
```

```
        if (L[i] <= R[j])
```

```
            arr[k++] = L[i++];
```

```
        else
```

```
            arr[k++] = R[j++];
```

```
    }
```


while (i < n1)

arr[k++] = L[i++];

while (j < n2)

arr[k++] = R[j++];

}

void mergeSort(int arr[], int left, int right)

{

if (left < right)

{

int mid = (left + right) / 2;

mergeSort(arr, left, mid);

mergeSort(arr, mid + 1, right);

mergeSort(arr, left, mid, right);

}

}

int main()

{

int n, i;

printf("Enter number of Employees:");

scanf("%d", &n);

int arr[n];

printf("Enter Salary : \n");


```
for(i=0; i<n; i++)  
    scanf("%d", &arr[i]);
```

```
mergeSort(arr, 0, n-1);
```

```
printf("Sorted Salaries : ");
```

```
for(i=0; i<n; i++)  
    printf("%d", arr[i]);
```

```
printf("In Lowest salary : %d", arr[0]);
```

```
printf("In Highest salary : %d", arr[n-1]);
```

```
printf("In Difference : %d", arr[n-1] - arr[0]);
```

```
return 0;
```

```
}
```



```

Mergesort3.c > main()
1  #include <stdio.h>
2  // Function to merge two sorted subarrays
3  void merge(int arr[], int left, int mid, int right)
4  {
5      int i, j, k;
6      int n1 = mid - left + 1;
7      int n2 = right - mid;
8
9      int L[n1], R[n2];
10
11     // Copy data into temporary arrays
12     for(i = 0; i < n1; i++)
13         L[i] = arr[left + i];
14
15     for(j = 0; j < n2; j++)
16         R[j] = arr[mid + 1 + j];
17
18     i = 0;
19     j = 0;
20     k = left;
21
22     // Merge both arrays
23     while(i < n1 && j < n2)

```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

enter number of employees: 7

enter salary:

45000

25000

70000

50000

30000

90000

60000

Sorted Salaries: 25000 30000 45000 50000 60000 70000 90000

Lowest Salary: 25000

Highest Salary: 90000

Difference: 65000

Topic 2 : Quick Sort

1. Race Timing Analysis.

```
#include <stdio.h>
```

```
void swap(int *a, int *b)  
{
```

```
    int temp = *a;
```

```
    *a = *b;
```

```
    *b = temp;
```

```
}
```

```
int partition(int arr[], int low, int high)  
{
```

```
    int pivot = arr[high];
```

```
    int i = low - 1;
```

```
    for (int j = low; j < high; j++)  
    {
```

```
        if (arr[j] < pivot)  
        {
```

```
            i++;
```

```
            swap(&arr[i], &arr[j]);
```

```
        }
```

```
    }
```

```
    swap(&arr[i + 1], &arr[high]);
```

```
    return i + 1;
```

```
}
```



```

void quickSort(int arr[], int low, int high)
{
    if (low < high)
    {
        int pi = partition(arr, low, high);

        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

```

```

int main()
{

```

```

    int n, i;

```

```

    printf("Enter number of Runners : ");
    scanf("%d", &n);

```

```

    int arr[n];

```

```

    printf("Enter race timings : \n");
    for (i = 0; i < n; i++)
        scanf("%d", &arr[i]);

```

```

    quickSort(arr, 0, n - 1);

```

```

    printf("Sorted Timings : ");
    for (i = 0; i < n; i++)
    {
        printf("%d", arr[i]);
    }

```

```

    return 0;
}

```



```

1  #include <stdio.h>
2  // Function to swap two elements
3  void swap(int *a, int *b)
4  {
5      int temp = *a;
6      *a = *b;
7      *b = temp;
8  }
9
10 // Function to partition the array
11 int partition(int arr[], int low, int high)
12 {
13     // Select the last element as pivot
14     int pivot = arr[high];
15     int i = low - 1;
16
17     // Place smaller elements before the pivot
18     for(int j = low; j < high; j++)
19     {
20         if(arr[j] < pivot)
21         {
22             i++;
23             swap(&arr[i], &arr[j]);

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

PS C:\Users\sanskriti shukla\Documents\C programs\Module
gcc Quicksort1.c -o Quicksort1 } ; if ($?) { .\Quicksor

```

6

55

49

62

58

45

51

Sorted Timings: 45 49 51 55 58 62

2. Organizing Library Book IDs.

```
#include <stdio.h>
```

```
void swap(int *a, int *b)
```

```
{
```

```
    int temp = *a;
```

```
    *a = *b;
```

```
    *b = temp;
```

```
}
```

```
int partition(int arr[], int low, int high)
```

```
{
```

```
    int pivot = arr[high];
```

```
    int i = low - 1;
```

```
    for (int j = low; j < high; j++)
```

```
    {
```

```
        if (arr[j] < pivot)
```

```
        {
```

```
            i++;
```

```
            swap(&arr[i], &arr[j]);
```

```
        }
```

```
    }
```

```
    swap(&arr[i+1], &arr[high]);
```

```
    return i+1;
```

```
}
```


3.

```

void quicksort(int arr[], int low, int high)
{
    if (low < high)
    {
        int pi = partition(arr, low, high);

        quicksort(arr, low, pi-1);
        quicksort(arr, pi+1, high);
    }
}

```

```

int main()
{
    int n, i;

    printf("Enter number of book IDs: ");
    scanf("%d", &n);

```

```

    int arr[n];

```

```

    printf("Enter book IDs: \n");
    for (i = 0; i < n; i++)
        scanf("%d", &arr[i]);

```

```

    quicksort(arr, 0, n-1);

```

```

    printf("Sorted Book IDs: ");
    for (i = 0; i < n; i++)
    {
        printf("%d", arr[i]);
    }
    return 0;
}

```



```

1  #include <stdio.h>
2  // Function to swap two elements
3  void swap(int *a, int *b)
4  {
5      int temp = *a;
6      *a = *b;
7      *b = temp;
8  }
9
10 // Function to partition the array
11 int partition(int arr[], int low, int high)
12 {
13     // Select the last element as pivot
14     int pivot = arr[high];
15     int i = low - 1;
16
17     // Move smaller elements before the pivot
18     for(int j = low; j < high; j++)
19     {
20         if(arr[j] < pivot)
21         {
22             i++;
23             swap(&arr[i], &arr[j]);

```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

PS C:\Users\sanskriti shukla\Documents\C programs\Mod

gcc Quicksort2.c -o Quicksort2 } ; if (\$?) { .\Quick

5

1025

1001

1050

1010

1005

Sorted Book IDs: 1001 1005 1010 1025 1050

3. Online Store Sales Ranking.

```
#include <stdio.h>
```

```
void swap(int *a, int *b)
```

```
{
```

```
    int temp = *a;
```

```
    *a = *b;
```

```
    *b = temp;
```

```
}
```

```
int partition(int arr[], int low, int high)
```

```
{
```

```
    int pivot = arr[high];
```

```
    int i = low - 1;
```

```
    for (int j = low; j < high; j++)
```

```
    {
```

```
        if (arr[j] > pivot)
```

```
        {
```

```
            i++;
```

```
            swap(&arr[i], &arr[j]);
```

```
        }
```

```
    }
```

```
    swap(&arr[i+1], &arr[high]);
```

```
    return i+1;
```

```
}
```

```
void quickSort(int arr[], int low, int high)
```

```
{
```

```
    if (low < high)
```

```
    {
```



```
int pi = partition(arr, low, high);
```

```
quicksort(arr, low, pi-1);
```

```
quicksort(arr, pi+1, high);
```

```
}  
}
```

```
int main()
```

```
{
```

```
int n, i;
```

```
printf("Enter number of sellers:");
```

```
scanf("%d", &n);
```

```
int arr[n];
```

```
printf("Enter sale values : \n");
```

```
for (i = 0; i < n; i++)
```

```
scanf("%d", &arr[i]);
```

```
quicksort(arr, 0, n-1);
```

```
printf("sorted code sales :");
```

```
for (i = 0; i < n; i++)
```

```
printf("%d", arr[i]);
```

```
printf("\n Top 3 : %d %d %d", arr[0], arr[1], arr[2]);
```

```
printf("\n Average : %d", (arr[0] + arr[1] + arr[2]) / 3);
```

```
return 0;
```

```
}
```



```

1  #include <stdio.h>
2  // Function to swap two elements
3  void swap(int *a, int *b)
4  {
5      int temp = *a;
6      *a = *b;
7      *b = temp;
8  }
9
10 // Function to partition the array in descending
11 int partition(int arr[], int low, int high)
12 {
13     // Select the last element as pivot
14     int pivot = arr[high];
15     int i = low - 1;
16
17     // Move larger elements before the pivot
18     for(int j = low; j < high; j++)
19     {
20         if(arr[j] > pivot)
21         {
22             i++;
23             swap(&arr[i], &arr[j]);

```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```

PS C:\Users\sanskriti shukla\Documents\C programs\Module 1>
gcc Quicksort3.c -o Quicksort3 } ; if ($?) { .\Quicksort3

```

6

1200

5000

3000

2500

7000

4500

Sorted Sales: 7000 5000 4500 3000 2500 1200

Top 3: 7000 5000 4500

Average: 5500